



Plan Your Journey. Experience More.

Project Documentation · v1.0.0 · September 2026

A full-stack, personalised travel-planning platform that turns a rough idea into a real, bookable-quality itinerary: multi-city routes with per-city dates, a day-by-day activity schedule, a live budget breakdown, and a public share link anyone can open.

1. Purpose of the Project

Planning a multi-city trip usually means juggling spreadsheets, notes apps and booking sites. GlobeTrotter solves this with a single workspace where a traveller can:

- **Discover** — destinations from a curated city catalogue (search, filters, cost index, popularity).
- **Plan a route** — multiple cities, each with its own arrive/leave dates, automatically re-clamped so a route can never become invalid.
- **Build an itinerary** — day by day with drag-and-drop scheduling, conflict detection and a curated activity catalogue per city.
- **Track a budget** — per-category and per-day breakdowns, a manual expense log and an optional spending limit with an over-budget alert.
- **View everything on a calendar** — month / week / day / timeline views with drag-to-reschedule.
- **Share a trip** — publicly via a link (no account needed to view) and let other travellers copy the trip into their own account.

In short: one platform for the whole journey — from inspiration to a complete, costed, shareable itinerary.

2. Technologies Used

Frontend (client/)

Technology	Version	Role
React	19	UI library
TypeScript	5.9	Type safety across the whole app
Vite	8	Dev server & build tool (proxies /api to the backend in dev)
Tailwind CSS	4	Utility-first styling, design tokens via CSS variables
React Router DOM	7	Client-side routing, route guards, layouts
TanStack React Query	5	Server-state management: caching, refetching, mutations
Axios	1.x	HTTP client with a session interceptor

React Hook Form + Zod	7 / 3	Form state and schema validation shared with the API
Radix UI	latest	Accessible headless primitives (dialogs, selects, tabs...)
dnd-kit	6 / 10	Drag-and-drop for itinerary reordering and calendar rescheduling
Recharts	2.x	Budget pie chart and cost-per-day bar chart
Framer Motion	13	Animations and page transitions
lucide-react	—	Icon set
sonner	2.x	Toast notifications
date-fns	4.x	Date formatting and calendar maths
cva + clsx + tailwind-merge	—	Component variant and class utilities

Backend (server/)

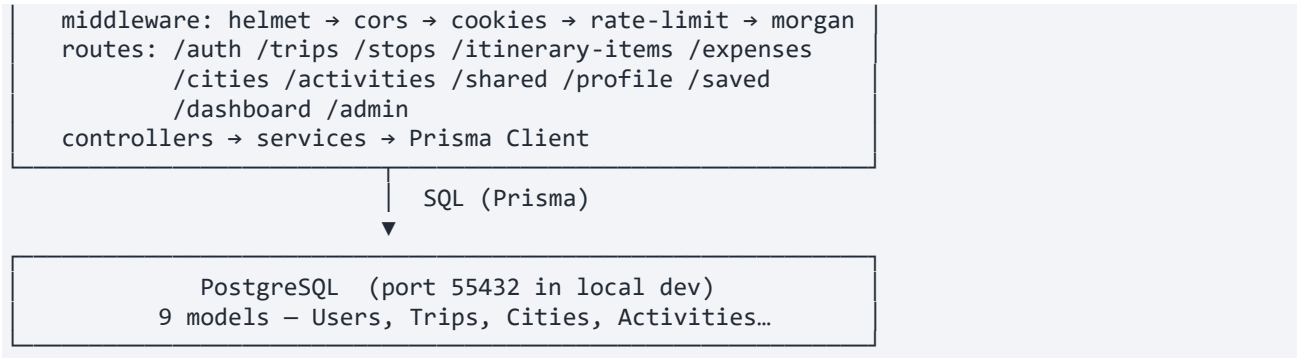
Technology	Version	Role
Node.js	≥ 20	Runtime
Express	5	REST API framework
TypeScript	5.9	Type safety
Prisma ORM	6	Type-safe database access, migrations and seeding
PostgreSQL	14+	Relational database
Zod	3	Request validation on every route
jsonwebtoken (JWT)	9	Session tokens
bcryptjs	3	Password hashing
Helmet	8	Security headers
CORS + cookie-parser	—	Cross-origin support and httpOnly cookie sessions
express-rate-limit	8	Brute-force protection (notably on auth routes)
Morgan	1.x	HTTP request logging
Vitest + Supertest	4 / 7	Integration tests against a real test database

Tooling & Infrastructure

Tool	Role
npm workspaces	Monorepo managing client/ and server/ from one root
concurrently	One command starts API + web dev servers together
tsx	TypeScript execution for dev server, seed and scripts
embedded-postgres	Zero-setup local PostgreSQL — npm run db:local downloads and runs a real Postgres binary on port 55432
dotenv	Environment configuration from a single root .env

3. System Architecture





Key Architectural Decisions

- **Monorepo with workspaces** — one npm install, one .env, shared tooling; client and server stay independently buildable.
- **Layered backend** — routes → controllers → services → Prisma. Business logic lives in services and is what the integration tests exercise.
- **Client state split** — TanStack Query owns server state (trips, cities, itinerary); React state/context owns UI state (theme, auth session, wizard steps).
- **Shared validation philosophy** — Zod on the API, with matching schemas on the client so form errors match server rules.
- **Single source of truth** — overview, itinerary, budget and calendar views all read the same API data, so they can never disagree.

4. Database Design

PostgreSQL via Prisma ORM. Full definitions in prisma/schema.prisma.

Models (9)

Model	Purpose	Notable design points
User	Accounts, preferences, role	Soft-delete via deletedAt so shared itineraries survive account deletion
PasswordResetToken	Forgot-password flow	Only the hash of the token is stored; tokens expire and are single-use
Trip	A journey: dates, sharing, budget limit	publicSlug enables share links; sourceTripId powers Copy Trip
City	Curated destination catalogue	Unique per (name, country); cost index + popularity for discovery
TripStop	One city visit within a trip	Ordered route with per-city arrive/leave dates
Activity	Curated things to do in a city	8 categories, duration, estimated cost, popularity
ItineraryItem	One scheduled day-plan entry	Can link an Activity or be free-form; supports time, notes, custom cost
Expense	Manual spending log	5 categories, Decimal(12,2) money
SavedDestination	City bookmarks per user	Unique (userId, cityId)

Enums

- **Role** — USER, ADMIN
- **ActivityCategory** — ADVENTURE, SIGHTSEEING, FOOD, CULTURE, SHOPPING, NATURE, ENTERTAINMENT, RELAXATION
- **ExpenseCategory** — TRANSPORT, ACCOMMODATION, ACTIVITIES, MEALS, MISCELLANEOUS

Data Conventions

- **IDs** — cuid() strings, safe in URLs, no enumeration.
- **Dates** — @db.Date — a trip can never shift a day because of timezones.
- **Clock times** — "HH:mm" strings — trips are local-time concepts.
- **Money** — Decimal(12, 2) — no float rounding in totals.
- **Delete rules** — cascades for trip-owned data; Restrict on City so catalogue entries with history can't vanish; SetNull on Activity links so past itineraries stay intact.

5. API Overview

Base URL: /api — all responses follow a consistent envelope ({ data } / { error }).

Router	Prefix	Access	Responsibility
Health	/health	Public	Liveness check
Auth	/auth	Public + session	Signup, login, logout, me, forgot/reset password
Trips	/trips	Owner/collaborator	CRUD, filters, sorting, pagination, copy
Stops	/stops	Owner	Add / reorder / remove cities on a route
Itinerary items	/itinerary-items	Owner	CRUD, reorder, conflict checking
Expenses	/expenses	Owner	Manual expense log CRUD
Cities	/cities	Public	Discovery: search, facets, filters, popular, detail
Activities	/activities	Public	Catalogue search with filters
Shared	/shared	Public	View a shared itinerary by slug; copy it
Profile	/profile	Session	Name, avatar, language, currency, preferences, password change, account deletion
Saved	/saved	Session	Save / unsave cities, list bookmarks
Dashboard	/dashboard	Session	Aggregated stats, upcoming trips, notifications
Admin	/admin	ADMIN role	User management and platform analytics

Note: Authentication is a JWT session token in an httpOnly cookie (invisible to client-side JavaScript), sent automatically by the browser; the client also carries an Axios interceptor to handle session expiry gracefully.

6. Feature Summary

Area	Highlights
------	------------

Trips	5-step creation wizard · multi-city routes with per-city dates · search, filters, sorting, pagination · grid & list views · delete with confirmation
Itinerary	Three-pane drag-and-drop builder · activity search with category/cost/duration filters · smart default scheduling · overlap & out-of-range conflict detection with "Add anyway" resolution
Budget	5-category split · pie + per-day bar charts · per-day statistics · manual expense log · optional limit with over-budget warning
Calendar	Month / week / day / timeline views · drag to reschedule or reorder · click-to-edit anything
Discovery	Searchable city catalogue · country/region/cost/popularity filters · rich city detail page · save/bookmark cities
Sharing	Public link per trip (no account needed to view) · view counter · Copy Trip into another account
Accounts	Signup / login / logout · forgot & reset password · profile, language & currency · theme (light/dark/system) · notification preferences · account deletion with anonymisation
Admin	Platform analytics · user management (role: USER / ADMIN)
UX	Fully responsive · dark mode · toasts · skeletons · error boundary · keyboard-accessible drag-and-drop

7. Security

- **Passwords** — hashed with bcrypt, never stored or logged in plain text.
- **Sessions** — JWTs in httpOnly cookies — invisible to client-side JavaScript (XSS-safe).
- **Reset tokens** — stored only as hashes, expiring and single-use.
- **Validation** — every request body/query validated with Zod before touching services.
- **Headers & limits** — Helmet security headers; rate limiting on sensitive routes.
- **CORS** — restricted to the configured client origin.
- **Secrets** — all credentials live in .env, which is gitignored; .env.example documents every variable without real values.

8. Project Structure

```

GlobeTrotter/
├── client/                                     # React SPA (Vite workspace)
│   └── src/
│       ├── components/                       # activity, auth, city, common, layout, trip, ui
│       ├── contexts/                         # AuthContext, ThemeContext
│       ├── hooks/                           # Reusable React hooks
│       ├── layouts/                         # AppShell, AuthLayout, PublicLayout, TripWorkspace
│       ├── lib/                             # api client, schemas, formatting, helpers
│       ├── pages/                           # Route pages (auth, trips, discover, admin...)
│       ├── services/                        # Typed API call layer
│       └── types/                           # Shared API types
├── server/                                   # Express REST API (workspace)
│   └── src/
│       ├── config/                          # env loading, constants
│       ├── controllers/                     # request handling, one per domain
│       ├── middleware/                      # auth, validation, errors, rate limiting
│       ├── routes/                          # one router per domain, mounted under /api
│       └── services/                        # business logic

```

utils/	# jwt, cookies, crypto, serializers
validators/	# Zod request schemas
tests/	# Vitest + Supertest integration tests
prisma/	
schema.prisma	# Database models & migrations
migrations/	# SQL migration history
seed.ts	# Demo users, 16 cities, 118 activities
scripts/	
dev-database.ts	# Zero-setup embedded PostgreSQL launcher
smoke-test.ts	# End-to-end API smoke check
.env.example	# Documented environment template
package.json	# Workspace root & shared scripts
README.md	# Full feature manual

9. Setup & Installation

Prerequisites: Node.js ≥ 20 and npm. No database installation needed — the project ships its own embedded PostgreSQL.

```
# 1. Install all dependencies (root + workspaces)
npm install

# 2. Create your env file and adjust if needed
cp .env.example .env

# 3. Generate the Prisma Client
npm run prisma:generate

# 4. Terminal 1 — start the local database (keep this terminal open)
npm run db:local

# 5. Terminal 2 — create tables and load demo data
npm run prisma:migrate
npm run seed

# 6. Start both servers
npm run dev
```

- **Web app** — <http://localhost:5173>
- **API** — <http://localhost:4000/api> (health check: [/api/health](http://localhost:4000/api/health))

Using Docker or a hosted Postgres (e.g. Neon) instead? Just point DATABASE_URL in .env at it and skip step 4 — no code changes needed.

10. Run Commands Reference

Command	What it does
npm run dev	Start API + web dev servers together
npm run db:local	Start the embedded PostgreSQL (port 55432)
npm run prisma:migrate	Apply schema migrations
npm run seed	Load demo users, cities and activities

npm run prisma:studio	Browse the database in a GUI
npm run typecheck	TypeScript check for server + client
npm run test	Run the backend integration test suite
npm run smoke	End-to-end API smoke test
npm run build	Production build (server + client)
npm run start	Run the built server
npm run db:reset	Wipe, re-migrate and re-seed the database

11. Testing

The backend has an integration test suite (Vitest + Supertest) that runs against a real test database — covering authentication, trips, itinerary, budget sharing and discovery.

```
npm run test      # run once
npm run test:watch # watch mode (inside server/)
```

Client quality gates: npm run typecheck (whole monorepo) and npm run build.

12. Demo Accounts

Created by npm run seed — development only.

Role	Email	Password
Demo traveller	demo@globetrotter.app	Password123!
Admin	admin@globetrotter.app	Password123!
Second traveller	priya@globetrotter.app	Password123!